Flaws of Basic Authentication and Advantages of JWT

This guide summarizes the discussion about Basic Authentication in a React + Spring Boot CRUD application and explains why JWT is preferred.

Part 1: Flaws of Basic Authentication

1. Username and password are sent with every authenticated request.

2. Base64 is encoding, not encryption; it can be easily decoded.

3. Repeated transmission of credentials increases exposure risk.

4. Clients often store the Basic Authorization header (Base64(username:password)) to avoid prompting users before every API call.

5. If the user's password changes, the stored Basic credentials become invalid and the user must authenticate again.

6. Logging out is handled mainly by the client because the server does not maintain login sessions for Basic Authentication.

7. The user's actual password acts as the authentication credential for every request.

8. No built-in token expiration or refresh mechanism.

9. Credentials do not contain claims such as roles or expiry.

10. Spring Security validates credentials on every request, usually requiring UserDetails lookup.

11. Less suitable for Single Page Applications and public APIs.

12. Limited support for modern session management such as per-device revocation.

Why Does the Client Store Basic Credentials?

HTTP is stateless. After login, the server does not remember the client. To avoid asking for the username and password before every API call, the React application commonly stores the Basic Authorization header in localStorage, sessionStorage, or memory and attaches it to each request.

If the password changes, the client continues sending the old Basic Authorization header until it is cleared. Spring Security decodes the old username and password, authentication fails, and the user must log in again.

Part 2: Advantages of JWT over Basic Authentication

1. Password is sent only once during login.

2. Future requests use a signed Bearer token instead of the password.

3. Tokens support configurable expiration.

4. Tokens can contain claims such as username, roles, issued time, and expiry.

5. Reduced exposure of user credentials.

6. Better suited for React, Angular, Vue, mobile apps, and REST APIs.

7. Supports stateless authentication while improving user experience.

8. Easier to extend with refresh tokens and token revocation strategies.

9. Password is not stored on the client after login.

10. Industry-standard approach for modern web applications.

Flow Comparison

Basic Authentication: Login -> React stores Basic header -> Every request sends Base64(username:password) -> Spring authenticates every request.

JWT: Login -> Spring validates credentials -> Generates JWT -> React stores JWT -> Every request sends Authorization: Bearer .

Conclusion

Basic Authentication is simple and useful for learning or internal systems but has several security and usability limitations. JWT addresses these by replacing repeated password transmission with signed tokens, enabling expiration, claims, and better support for modern frontend applications.